Doc. No.:	P1221R1
Date:	2018-10-03
Reply To:	Jason Rice ricejasonf@gmail.com
Title:	Parametric Expressions
Audience:	Evolution Working Group

Parametric Expressions

Try it out: https://godbolt.org/z/Zo9ozy

Abstract

The purpose of this paper is to propose a hygienic macro that transforms expressions during semantic analys template instantiation. This can replace macros in certain cases and enable more idiomatic code, operations have potentially faster compile times versus function templates.

There are two types of transformations, *transparent* and *opaque*, the latter of which supports multiple stateme cleanups via RAII. Both are hygienically scoped.

Consider the following declaration syntax:

```
using add(auto a, auto b) {
  return a + b;
}
```

Here is a function-like declaration that has the same behaviour as a function without creating a function type, still has its own scope.

```
int main() {
  int i = 40;
  int x = add(i++, 2);
}

// the same as

int main() {
  int i = 40;
  int x = ({
    auto&& a = i++;
    auto&& b = 2;
    a + b; // result of expression
  });
}
```

> By default each input expression is bound to a variable resulting in evaluation that is consistent with norma call expressions. User controlled evaluation is addressed in *Using Parameters*.

Function templates as we have them now are loaded with features such as overloading, constraints, type dec SFINAE, ect.. When called not only do we get an expensive template instantiation but also potentially large s extraneous code generation all for functions we wouldn't call otherwise unless we expected the optimizer to i anyways.

With this tool we want to give programmers the ability to allow this inlining at an earlier stage, but since the in guaranteed and in the context of where it is called we can do things not currently possible with normal functio creating a special type for each invocation.

Constexpr Parameters

Since the invocation is inlined we easily get constexpr parameters without generating a complicated symbol f prototype.

```
using to_int_c(constexpr auto x) {
  return std::integral_constant<int, x>{};
}

int main() {
  constexpr int x = 42;
  std::integral_constant<int, 42> foo = to_int_c(42);
}
```

Using Parameters

With `using` as a specifier, we allow the user to take control of evaluation with macro-like semantics.

```
using if_(using auto cond, using auto x, using auto y) {
  if (cond)
    return x;
  else
    return y;
}

int main() {
  if_(
    true,
    print("Hello, world!"),
    print("not evaluated so nothing gets printed")
  );
}
```

This has some compelling use cases for creating idiomatic interfaces:

```
// customized conjunction and disjunction with short-circuit evaluation

Foo* y = x || new Foo();

auto y = do_something() && maybe_do_something_else();
```

```
// constexpr ternary expression

auto x = constexpr_if(true, int{42}, std::make_unique<int>(42));
```

```
// debug logging that eliminates evaluation in production (like macros)

namespace log {
  struct null_stream {
    using operator<<(using auto) {
      return null_stream{};
    }
  };

  using warn() {
    if constexpr (log_level == Warning)
      return std::wcerr;
    else
      return null_stream{};
  }
}

foo > 42 && log::warn() << "Foo is too large: " << foo << '\n';
```

Concise Forwarding

With this feature we can implement a forwarding operator that is more concise than the existing `std::forwa`

```
using fwd(using auto x) {
  return static_cast<decltype(x)>(x);
```

```
}

auto old_f = [](auto&& x) { return std::forward<decltype(x)>(x); };
auto new_f = [](auto&& x) { return fwd(x); };
```

Operations on Parameter Packs

We can allow unexpanded parameter packs as inputs and avoid creating tuples in many cases.

```
template <typename ...Xs>
auto foo(Xs... xs) {
  auto fifth_element = pack::get<5>(xs);

  auto bar = pack::apply(f, pack::drop_front(xs));
}
```

The ability to return a pack can reduce the need for additional helper functions for things like unpacking index tuples:

```
{
  int sum = (int_seq(5) + ...);
}
{
  std::array<int, 5> xs = {1, 2, 3, 4, 5};
  int sum = (xs[int_seq(5)] + ...);
}
{
  int xs[] = {1, 2, 3, 4, 5};
  int sum = (xs[int_seq(5)] + ...);
}
{
  constexpr std::tuple xs{1, 2, 3, 4, 5};
  constexpr int sum = (unpack(xs) + ...);
}
```

Don't believe it? Try it here: https://godbolt.org/z/Zo9ozy

Friendlier Interfaces

Some people don't like the angle bracket soup that templates can bring when interfaces require constexpr fri Here we can hide it with a simple wrapper (taken from here):

```
#include "ctre.hpp"

using match(constexpr auto x, using auto sv) {
    static constexpr auto pattern = ctll::basic_fixed_string{ x };
    constexpr auto re = ctre::re<pattern>();
    return re.match(sv);
}


int main() {
    return match("Hello.*", "Hello World");
}
```

As a Member of a Class

Parametric expressions can be used as a member in conjunction with operator overloading to create an invo

```
  struct id_fn {
    using operator()(using auto self, auto x) {
      return x;
    }
  };
```

```
  id_fn{}(42);

  static_assert(std::is_invocable<id_fn, int>::value);
```

Transparent Transformations

Specific conditions allow us to generate an expression that does not require RAII and therefore allows the co substitute the call expression directly which can also be used in SFINAE and other contexts where variable d and statements are not welcome.

```
using funcify(using auto f) {
  return [](auto&& ...x)
    noexcept(noexcept(decltype(f(std::forward<decltype(x)>(x)))))
    -> decltype(f(std::forward<decltype(x)>(x)...))
    { return f(std::forward<decltype(x)>(x)...); };
}

struct make_array_impl {
  using operator()(using auto ...x) {
    return std::array<std::common_type<decltype(x)...>, sizeof...(x)>{x...};
  }
};

constexpr auto make_array = funcify(make_array_impl{});
```

When a RAII scope is needed these will be referred to as opaque transformations.

Comparison with Other Proposed Features

One of the primary benefits of this proposed feature is the compile-time performance and the fact that it has z the type system. That with operations on packs, there are no other proposals addressing this. However this p feature does have overlap with other proposed features concerning constexpr parameters and conditional ev

The paper `constexpr` *Function Parameters* (P1045R1) by David Stone proposes the `constexpr` specifier t normal function parameter declarations effectively making the parameter `constexpr`. The advantage that it l Parametric Expressions proposal is that it allows function overloading. While this is still highly desirable, it co costs which will affect compiler throughput. First, it requires overload resolution to check if an argument expre used in a constant expression. Second, each constexpr parameter would affect the type creating a new funct symbols that would have to be similar to anonymous classes for each permutation of input arguments. If this feasible, there is no conflict and the `constexpr` parameters proposed by this paper are consistent with Davi proposal and the feature proposed in this paper would be a useful augment for when overloading is not need

The paper *Towards A (Lazy) Forwarding Mechanism for C++* (P0927R0) by James Dennet and Geoff Romer sets out to add a feature to enable conditional evaluation of function parameters. The paper suggests the use similar to a lambda expression for the parameter declaration to specify laziness of evaluation which is a majo convention as far as variable declarations are concerned. This paper is in agreement with the motiviations sta solutions are in contrast. Having the same issues with creating a function prototype, the paper addresses the on the type system suggesting that lazy parameters be added to the type system which is high impact, comp opinion of this paper, unnecessary. This paper, however, is proposing the use of an existing keyword in the de specifier which is more in keeping with existing conventions with parameter declarations, and there is no imp system which makes it a better solution.

Notable Concerns

Why not go for broke and jump straight into granular AST node manipulation?

Some have suggested that we add something more akin to raw AST generation as is found in Rust macros. \ would be a powerful superset of the feature this paper is proposing, it comes with some issues. One, this wou interface **significantly more complicated** and inaccessible to typical users. It also comes with the same pro evaluation that preprocessor macros have in that it easily allows mistakes that result in duplicate evaluation c operations.

The interface proposed with *Parametric Expressions* is such that it defaults to consistent evaluation of input e binding them to an actual variable declaration in the parameter list. If the author of a function wants to take co evaluation they can do so via the `using` parameter declaration specifier and thus opt in explictly and without AST syntax.

That said, the advent of Reflection and Metaclasses may open the door to allowing more refined control over this paper's stance is that this feature is **forwards compatible** with the possibilty of adding `reflexpr` param similar construct to enable this superset.

Invocable and the Dangers of `using` Parameters

As previously stated, `using` parameters add macro-like semantics. While the pitfalls of macros are well unde is an edge case that is worth mentioning:

```
struct eval_twice_fn {
  using operator()(using auto x) {
    return std::pair(x, x);
  }
} inline constexpr eval_twice{};

auto result1 = eval_twice(my::string("foo"));
auto result2 = std::invoke(eval_twice, my::string("foo"));
```

Here the semantics are changed when used with `std::invoke`. Since `std::invoke` is a normal function, it and binds it to a parameter that is perfectly forwarded. The forwarding expression would then be passed to `e` and the result of the input would get moved twice. With the call expression the result is a pair of two distinct s containing a pointer to their own "foo".

`std::Invokable` does not require that the call to `std::invoke` be equality preserving on its arguments so t permissible. Regardless, this paper's stance is to allow these types of edge cases, and leave it to the discreti as the user is opting in explicitly via the `using` specifier. Attempts to prevent these would involve complex ru inadvertantly rule out desirable use cases.

Impact on the Existing Standard

While there is zero impact on existing features in C++, the following additions are required:

1. Parsing the `using` keyword followed by an identifier or operator name followed by a left parentheses wo the declaration of a parametric expression disambiguating from other valid uses of the `using` keyword f of declarations.
2. Allow the `constexpr` specifier in parameter declarations in the context of a parametric expression parar declaration.
3. The `using` keyword is used as a declaration specifier for parameter declarations in the context a param expression parameter list declaration.
4. The `auto` keyword is used as placeholder for a constraint for parametric expression parameters and it i promotes parameters as `auto&&`. This is more concise but a slight deviation from what C++ programme expect.

Implementation Experience

The proposed language feature has been implemented in a fork of Clang here: reference implementation

From this the following are rules to specify its workings in simple terms:

1. Declaration or invocation does not create a type.
2. The input parameter list may contain only one parameter pack in any position.
3. Input parameters do not specify a type or have type specifiers.
    - The **`auto` is a placeholder for a constraint** which is meant for forwards compatibility with languag

4. Parameter declarations can have `constexpr` or `using` specifiers which change semantics:
    - With none of these specifiers the parameters is an forwarding reference initialized with the input exp auto&&).
    - The `constexpr` specifier enables value initialization with an expression that must be a constant-exp constexpr auto)
    - The `using` specifier does not bind the input expression to a variable, but the input expression is su everywhere the name is used. (like a macro)
5. Parameters that bind to a variable are specified to evaluate in the order of their declaration from left to ri evaluation of the compound statement.
6. There are two types of transformations: *Opaque* and *Transparent*
    - Transparent transformations occur when there are no variable declarations (except `using` vars), ar single statement that is a return statement. The resulting expression substitutes the call expression
    - All others are opaque transformations and create a RAII scope where their return statements must b expressions. Typical RVO/NRVO rules for return values apply as they do in functions.
7. Transparent transformations may have "parts" of expressions as inputs and outputs.
    - This includes expressions that do not refer to values such as template names and others.
    - An unexpanded parameter pack is allowed as an output but is expanded before input.
8. An Opaque transformation's return statements should follow the same rules for function return type dedu `decltype(auto)`.
9. Recursion is not feasible, but it is possible to achieve with templates or dependent input expressions.
10. Operator overloading is supported along with ADL, and it creates an ambiguous call if there are other via present.
11. When defined as a member of a class, invocations include the base expression as the first parameter. (i parameter)
    - The reason for this is that the type is deduced and it is not compatible with the way `this` works in f definitions. Also it gives the user flexibility to define the self parameter as `using` or `constexpr`.
12. As a member, the static keyword can be used to simply make it a free entity that is local to the owning cl
    - This means there is no self param.
13. Operator overloads can be declared as `static`.

Summary

With this we have a facility for writing more idiomatic and concise code with operations on packs all without p stack or extraneous type generation. Please consider this language feature for a future standard of C++.

Thanks for looking at this!

Jason Rice

Acknowledgements

Special thanks to Louis Dionne and Miguel Ojeda who provided feedback and suggestions as well as others feedback on the mailing list.